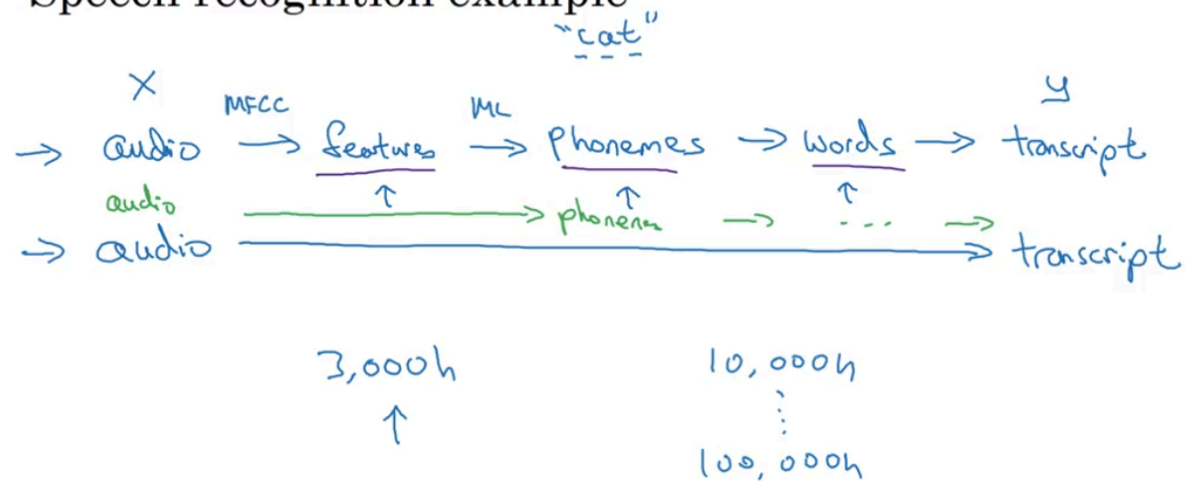


End - to - end Deep Learning

What is End-to-End deep learning

What is end-to-end learning?

Speech recognition example



Here's a **simple, easy-to-understand** note version:

End-to-End Deep Learning (Main Idea)

- Traditional systems:
 - Solve problem in **multiple steps (pipeline)**
- End-to-end learning:
 - Use **one neural network**
 - Go directly from **input** → **output**

Example: Speech Recognition

Traditional Approach (Many Steps)

1. Extract features (e.g., MFCC)
2. Detect phonemes (basic sounds)
3. Combine phonemes → words
4. Combine words → full transcript

End-to-End Approach

- Input: audio
- Output: transcript

👉 One model learns everything **automatically**

Why It's Powerful

- No need for **manual feature engineering**
- Simpler pipeline
- Model learns **best features by itself**

Challenge: Needs a Lot of Data

- **Small data** (e.g., 3,000 hours)

→ Traditional pipeline works better

- **Large data (e.g., 10,000+ hours)**

→ End-to-end works very well

Intermediate Approach

- Not fully end-to-end
- Example:
 - Input audio → predict phonemes (skip feature engineering)

👉 Step-by-step move toward end-to-end

Key Insight

- End-to-end = simpler system
- But requires **large dataset**

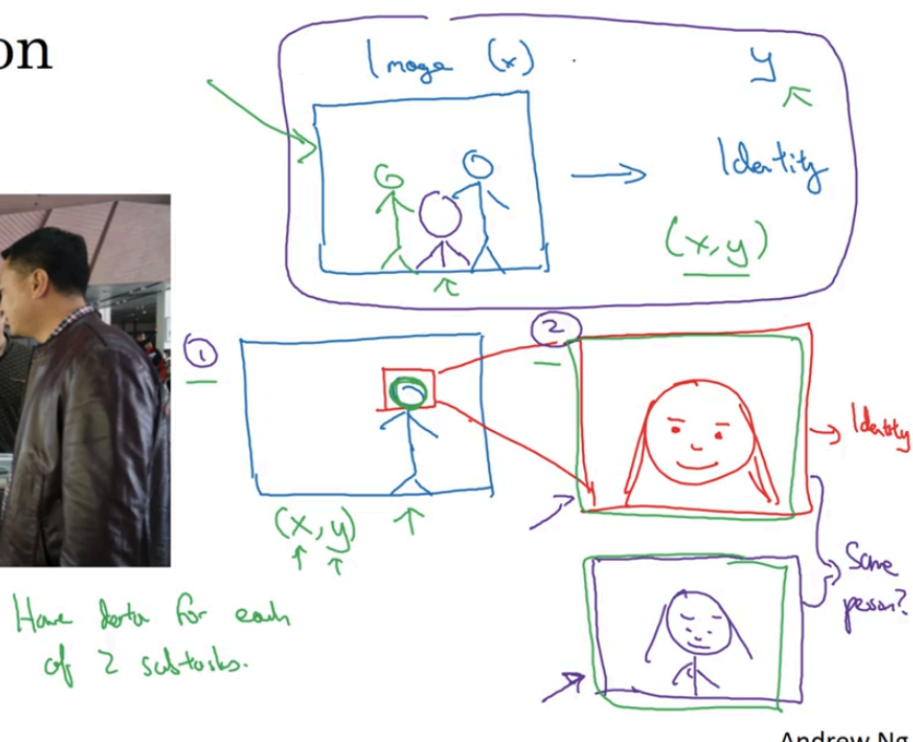
Takeaway

- Use end-to-end when:
 - You have **a lot of data**
- Otherwise:
 - Traditional pipeline may be better

Face recognition



[Image courtesy of Baidu]



Here's a **simple, easy-to-understand note version**:

Example: Face Recognition Turnstile

- System lets people enter by **recognizing their face**
- No need for RFID card → just walk in

Naive (End-to-End) Approach ❌

- Input: full image from camera
- Output: person's identity

👉 Problem:

- Face position changes (angle, distance, size)
- Hard for model to learn directly

Better Approach: Multi-Step Pipeline

Step 1: Face Detection

- Find **where the face is** in the image

Step 2: Face Recognition

- Crop the face
- Predict **who the person is**

How Recognition Works

- Model compares **2 images**
- Output: same person or not

👉 For real system:

- Compare with database (e.g., 10,000 employees)

Why Multi-Step Works Better

1. Simpler Tasks

- Task 1: find face
- Task 2: recognize face
 - Easier than doing everything at once

2. More Data Available

- **Face detection:**
 - Lots of labeled data (face positions)
- **Face recognition:**
 - Millions of face images available
- **End-to-end (image → identity):**
 - Very little data

Key Insight

- If:
 - You don't have enough data for end-to-end
- Then:
 - Break problem into **smaller steps**

Takeaway

- End-to-end is powerful
- But not always best

👉 Use multi-step approach when:

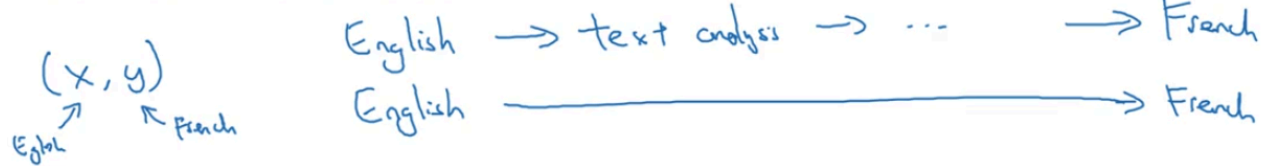
- Tasks are complex
- Data for full problem is limited
- But data for sub-tasks is large

Final Idea

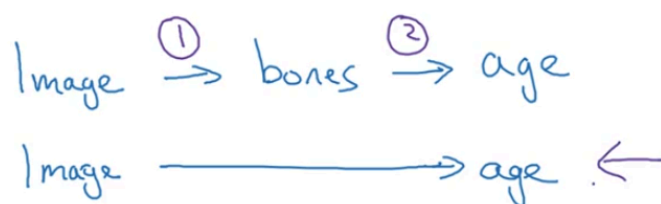
- Best approach depends on:
 - **data availability**
 - **problem complexity**

More examples

Machine translation



Estimating child's age:



Andrew Ng

Here's a **simple, clean note version**:

Example 1: Machine Translation

Traditional Approach

- Many steps:
 - Analyze text
 - Extract features
 - Process through pipeline
 - Output translation

End-to-End Approach

- Input: English sentence
- Output: French sentence

👉 Works well because:

- There is **a lot of data** (English–French pairs)

Example 2: Age from X-ray

Goal:

- Input: X-ray of child's hand
- Output: child's age

Traditional (Multi-Step) Approach ✓

1. Detect/segment bones
2. Measure bone lengths
3. Use lookup table → estimate age

👉 Works well with **limited data**

End-to-End Approach ❌

- Input: X-ray
- Output: age directly

👉 Problem:

- Needs **a lot of data**
- Currently **does not work as well**

Key Insight

- End-to-end works best when:
 - You have **large labeled datasets** ($X \rightarrow Y$)
- Multi-step works better when:
 - Data is limited
 - Problem can be broken into simpler parts

Takeaway

- End-to-end:
 - Simple system
 - Powerful
 - Needs **a lot of data**
- Multi-step:
 - More complex
 - But works better with **less data**

Final Idea

- End-to-end is **not always the best choice**
- Choose based on:
 - **data availability**
 - **problem structure**

Whether to use End-to-end deep learning

Pros and cons of end-to-end deep learning

Pros:

- Let the data speak $x \rightarrow y$ → "phonemes"
cat
- Less hand-designing of components needed

Cons:

- May need large amount of data $x \rightarrow y$ (x,y)
- Excludes potentially useful hand-designed components

Data.

Hand-design.

Andrew Ng

Here's a **simple, easy-to-understand note version**:

End-to-End Deep Learning: Pros & Cons

✓ Advantages

1. Lets the Data Speak

- Model learns directly from $\mathbf{X} \rightarrow \mathbf{Y}$
- No need to follow human-designed steps
- Can discover **better patterns automatically**

👉 Example:

- No need to force model to use **phonemes**
 - Model may learn a **better representation**
-

2. Less Manual Work

- No need to:
 - design features
 - build complex pipelines

👉 Simpler system design

✗ Disadvantages

1. Requires a Lot of Data

- Need many **(X, Y) pairs**
- Hard when full data is limited

👉 Example:

- Easy to get data for sub-tasks
 - Hard to get data for full end-to-end task
-

2. Ignores Useful Human Knowledge

- No hand-designed components
- May lose useful insights from experts

👉 Important idea:

- Learning = **data + human knowledge**
-

Key Insight

- If:
 - **Lots of data** → End-to-end works well
 - If:
 - **Little data** → Hand-designed steps help more
-

Trade-off

- End-to-end:
 - Flexible, data-driven
- Traditional pipeline:
 - Uses human knowledge

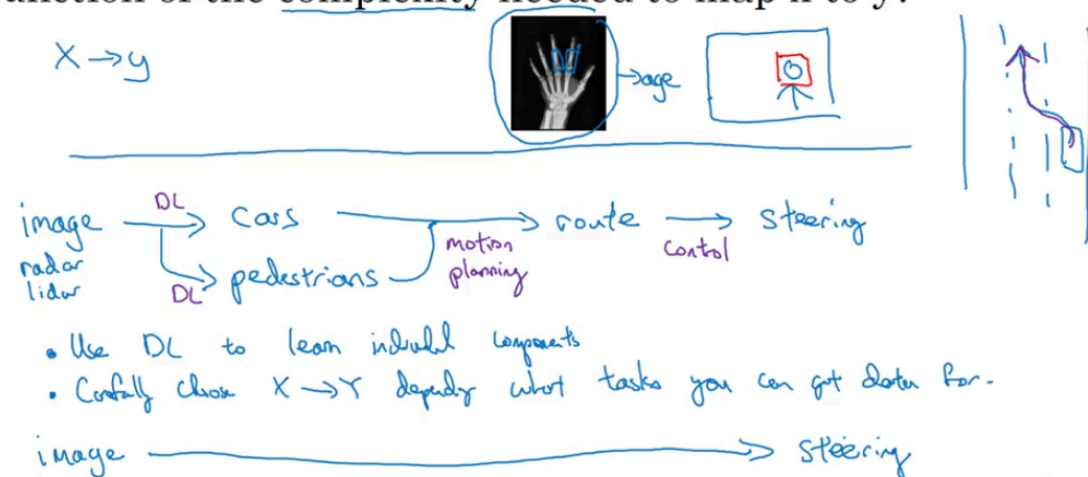
👉 Both can help or hurt depending on situation

Takeaway

- Use end-to-end when:
 - You have **large datasets**
- Use multi-step approach when:
 - Data is limited
 - Human knowledge is valuable

Applying end-to-end deep learning

Key question: Do you have sufficient data to learn a function of the complexity needed to map x to y ?



Andrew Ng

Here's a **simple, clear note version**:

Key Question for End-to-End Learning

👉 Do you have enough data to learn $X \rightarrow Y$ directly?

- If the task is **simple** → need less data
 - If the task is **complex** → need a lot more data
-

Examples

- **Simple tasks:**
 - Detect face in image
 - Detect bones in X-ray
 - Can work with less data
 - **Complex tasks:**
 - Image → person identity
 - X-ray → age
 - Need much more data
-

Autonomous Driving Example

Multi-Step Approach (Better in practice)

1. Input: image / sensor data
2. Detect:
 - cars

- pedestrians

3. Plan route (motion planning)

4. Control:

- steering
 - acceleration / braking
-

End-to-End Approach (Less practical now)

- Input: image
- Output: steering directly

👉 Problem:

- Too complex
 - Not enough data
 - Hard to train well
-

Key Insight

- Use deep learning for **parts you have data for**
 - Don't force everything into end-to-end
-

Final Takeaway

- End-to-end works when:
 - You have **a lot of data**
 - Task is manageable
 - Multi-step works better when:
 - Task is complex
 - Data is limited
-

Big Idea

- Choose **what to learn ($X \rightarrow Y$)** carefully
- Combine:
 - Deep learning
 - Traditional methods

👉 Build the **best system, not just the simplest one**